GANisme · texte de la soutenance

Kahina ABBAD : diapositives 1 à 12 · Pape Ahmed Fall : diapositives 13 à 24 · environ 15 minutes

Partie de Kahina



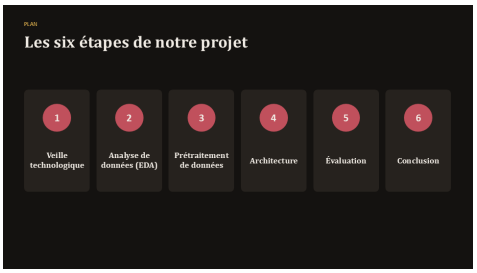
1. GANisme

Kahina

Bonjour à tous. Notre projet s'appelle GANisme. Notre objectif : créer une intelligence artificielle qui peint des tableaux qui n'existent pas.

Pour cela, nous avons utilisé un GAN, un réseau antagoniste génératif. Nous l'avons entraîné sur des milliers de peintures de la Web Gallery of Art. Les tableaux que vous voyez ici font partie de nos données.

Voyons d'abord comment notre présentation est organisée.



2. Les six étapes de notre projet

Kahina

Notre présentation suit six étapes.

Je vais vous présenter la veille technologique, l'analyse de nos données et le choix de nos images.

Pape vous présentera ensuite le prétraitement, l'architecture du modèle, son évaluation et notre conclusion.

Commençons par la veille : qu'est-ce qu'un GAN ?



3. Deux réseaux en compétition

Kahina

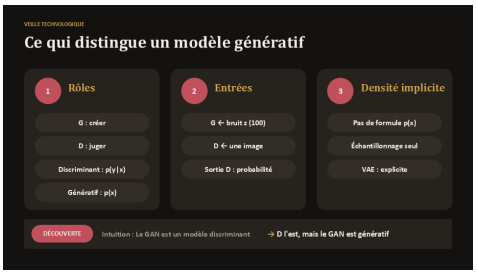
Le GAN a été inventé en 2014 par Ian Goodfellow.

Son but : apprendre à créer de nouvelles images qui ressemblent à des exemples. Il n'a besoin d'aucune étiquette : on parle d'apprentissage non supervisé.

Il repose sur deux réseaux qui s'affrontent. Le générateur part d'un bruit au hasard et fabrique une image. Le discriminateur reçoit une vraie peinture ou une image fabriquée, et doit dire si elle est vraie ou fausse.

À chaque erreur, le générateur apprend à mieux tromper le discriminateur. Les deux progressent ensemble.

Mais pourquoi dit-on que le GAN est un modèle génératif ?



4. Ce qui distingue un modèle génératif

Kahina

Il existe deux grands types de modèles.

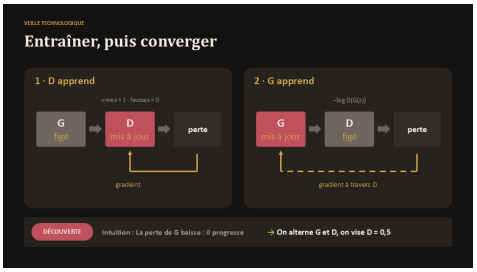
Un modèle discriminant apprend à classer : par exemple, dire si une image est un portrait ou un paysage.

Un modèle génératif, lui, apprend à quoi ressemblent les données. Il peut donc en créer de nouvelles.

Dans un GAN, le discriminateur classe, mais le GAN dans son ensemble crée. À la fin, c'est le générateur que l'on garde.

Le générateur reçoit 100 nombres tirés au hasard. Il sait produire des images, mais il ne sait pas dire si une image est probable ou non.

Voyons maintenant comment ces deux réseaux apprennent ensemble.



5. Entraîner, puis converger

Kahina

L'entraînement se fait en deux temps, qui se répètent.

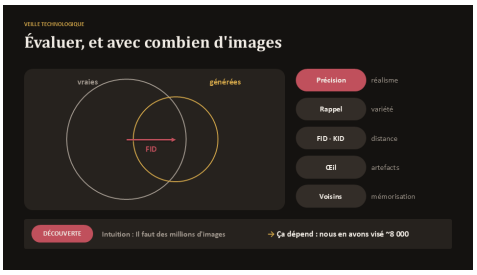
D'abord, le discriminateur apprend à reconnaître les vraies et les fausses images. Le générateur ne bouge pas.

Ensuite, c'est le générateur qui apprend à tromper le discriminateur. Cette fois, c'est le discriminateur qui ne bouge pas.

Le but n'est pas qu'un des deux gagne, mais d'atteindre un équilibre : le discriminateur ne sait plus faire la différence.

Le risque principal s'appelle le mode collapse : le générateur produit toujours la même image. C'est pourquoi nous avons regardé les images générées à chaque étape.

Mais comment savoir si les images sont réussies ?



6. Évaluer, et avec combien d'images

Kahina

Pour juger un GAN, un seul indicateur ne suffit pas.

Le FID mesure l'écart entre les vraies images et les images générées. Plus il est bas, plus elles se ressemblent. Le KID fonctionne sur le même principe.

Nous regardons aussi les images nous-mêmes, et nous vérifions que le modèle ne recopie pas les tableaux qu'il a vus.

Le nombre d'images nécessaires dépend de leur variété. Nous avons visé environ 8 000 peintures.

Il existe aussi plusieurs familles de GAN. Voyons lesquelles.



7. Variantes, et CNN contre GAN

Kahina

Les GAN produisent des images nettes, et très vite. Mais leur entraînement est instable.

Depuis 2014, de nombreuses versions sont apparues. Par exemple, CycleGAN transforme une photo en tableau de Monet, et StyleGAN crée des visages très réalistes.

Nous avons choisi le DCGAN. C'est un GAN qui utilise des réseaux convolutifs, des réseaux spécialisés dans les images.

Pour finir cette veille, parlons de l'IA générative en général.



8. ChatGPT, risques et intérêt

Kahina

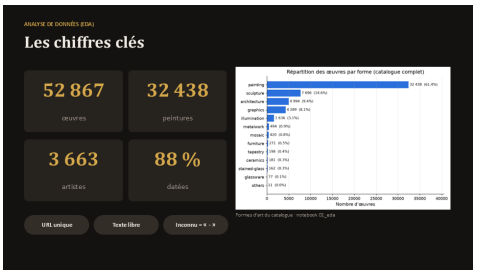
On pense souvent à ChatGPT quand on parle d'IA générative. Mais ce n'est pas un GAN : c'est un modèle de langage, développé par OpenAI.

Ces outils présentent des risques : les fausses vidéos, la désinformation, les biais ou les droits d'auteur.

Mais ils sont aussi utiles : compléter des données rares, protéger des données personnelles, ou aider les artistes.

Pour nous, c'est utile à une condition : qu'une image générée soit toujours présentée comme telle.

Passons maintenant à nos données.



9. Les chiffres clés

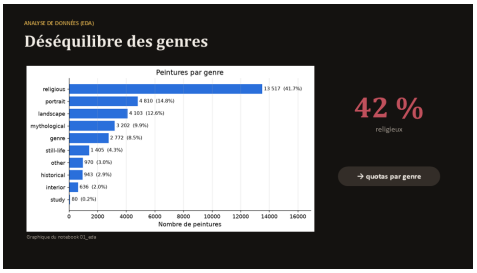
Kahina

Nos données viennent de la Web Gallery of Art.

Le catalogue compte près de 53 000 œuvres. Nous avons gardé uniquement les peintures : un peu plus de 32 000, réalisées par plus de 3 600 artistes.

Avant de récupérer la moindre image, nous avons étudié ces informations. Elles étaient écrites librement, par exemple « vers 1530 ». Nous les avons transformées en données exploitables : l'année, la taille et la technique.

Cette analyse nous a réservé une première surprise.



10. Déséquilibre des genres

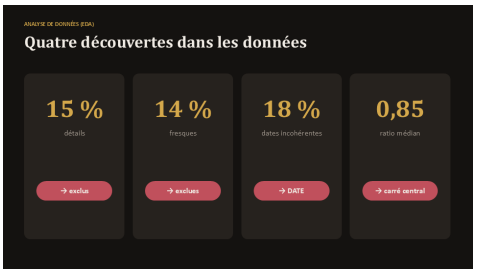
Kahina

Près de la moitié des peintures, 42 %, ont un sujet religieux.

Si nous avions pris les images au hasard, notre GAN aurait surtout appris à peindre des scènes bibliques.

Nous avons donc choisi autant d'images pour chaque genre : portraits, paysages, natures mortes, et ainsi de suite.

Nous avons aussi trouvé d'autres problèmes dans les données.



11. Quatre découvertes dans les données

Kahina

Nous avons repéré quatre problèmes.

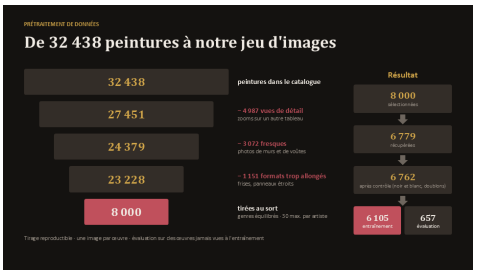
Premièrement, 15 % des images sont des zooms sur un autre tableau. Ce sont presque des doublons : nous les avons retirées.

Deuxièmement, 14 % sont des fresques, photographiées sur des murs. Elles sont trop différentes : nous les avons retirées aussi.

Troisièmement, la colonne des périodes était fausse dans 18 % des cas. Nous avons utilisé la vraie date.

Enfin, les tableaux sont souvent plus hauts que larges, alors que le GAN produit des images carrées. Nous les avons donc coupés en carré, au centre.

Voici comment ces décisions nous ont menés à notre jeu d'images.



12. De 32 438 peintures à notre jeu d'images

Kahina

Ce schéma se lit de haut en bas.

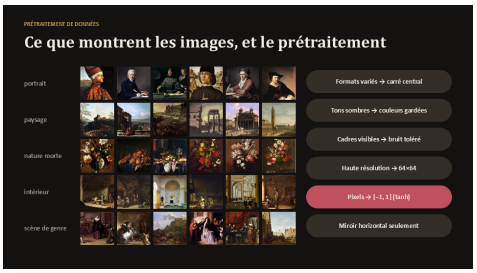
Nous partons de 32 438 peintures. Nous retirons les zooms, les fresques et les formats trop allongés. Il en reste 23 228.

Parmi elles, nous en choisissons 8 000 au hasard, avec autant d'images par genre et 30 au maximum par artiste.

À droite, le résultat : nous avons récupéré 6 779 images. Après nos contrôles, nous en gardons 6 105 pour entraîner le modèle et 657 pour le tester. Le modèle ne voit jamais ces 657 images pendant son apprentissage.

Je laisse maintenant la parole à Pape, qui va vous expliquer comment nous avons préparé ces images.

Partie de Pape



13. Ce que montrent les images, et le prétraitement

Pape

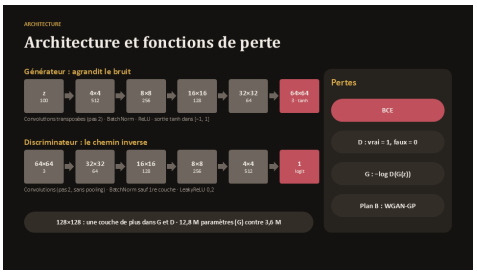
Merci Kahina. Voici ce que montrent nos images, et ce que nous en avons déduit. Les formats varient : nous recadrons en carré au centre. Les tons sont sombres : nous les gardons, c'est le style à apprendre. La résolution d'origine suffit largement pour du 64×64. Les pixels sont ramenés entre −1 et 1, pour correspondre à la sortie du générateur, et la seule augmentation est le miroir horizontal.



14. Un bug révélé par les visuels

Pape

Ce graphique raconte une erreur évitée. Pour repérer les doublons, une empreinte d'image signalait 1 615 paires. En les regardant, nous avons vu que c'étaient presque toujours des paysages différents, avec le même ciel clair et le même sol sombre. Une vérification pixel par pixel n'a confirmé que 3 vrais doublons. Sans elle, nous aurions supprimé des centaines de paysages sans le voir.



15. Architecture et fonctions de perte

Pape

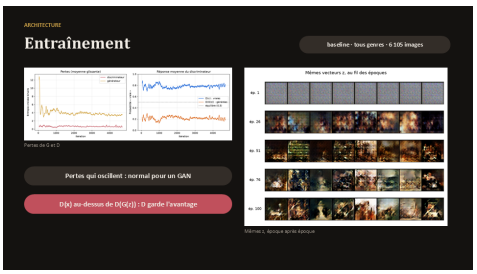
Notre générateur part de 100 nombres et agrandit l'image couche par couche, de 4×4 jusqu'à 64×64, avec une sortie tanh. Le discriminateur fait le chemin inverse jusqu'à un seul nombre. La perte est l'entropie croisée binaire : le discriminateur apprend « vrai = 1, faux = 0 », le générateur cherche à faire répondre « vrai ».



16. Un DCGAN, avec les réglages du papier

Pape

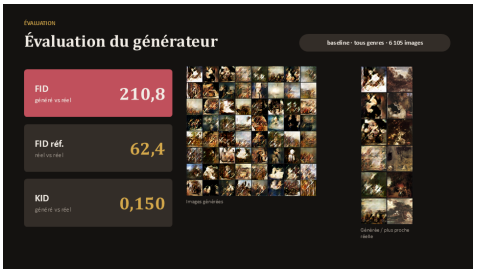
Nous avons choisi un DCGAN : c'est la référence en 64×64, simple à diagnostiquer et entraînable sans carte graphique. Nos hyperparamètres sont ceux de l'article original : Adam, un taux d'apprentissage de 0,0002, un β1 de 0,5, des lots de 128 images. Notre baseline, entraînée sur tous les genres, a tourné 100 époques sur 6 105 images.



17. Entraînement

Pape

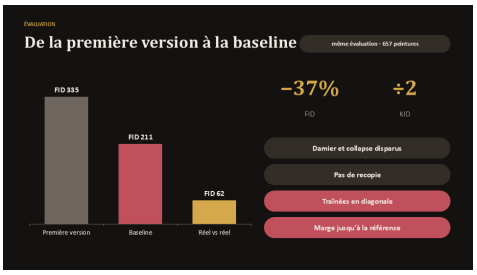
Voici l'entraînement de notre baseline. Les pertes oscillent, ce qui est normal pour un GAN, et le discriminateur garde l'avantage. À droite, les mêmes bruits époque après époque : on part du gris, on voit des motifs apparaître, puis une vraie matière picturale.



18. Évaluation du générateur

Pape

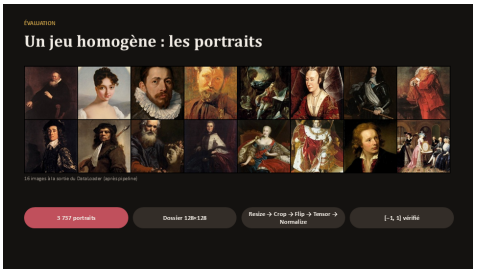
Notre baseline obtient un FID de 211. Pour l'interpréter, nous avons calculé un FID de référence entre deux ensembles de vraies peintures : 62, le meilleur score possible ici. Il reste donc de la marge. À droite, chaque image générée est différente de la peinture la plus proche : le modèle crée, il ne recopie pas.



19. De la première version à la baseline

Pape

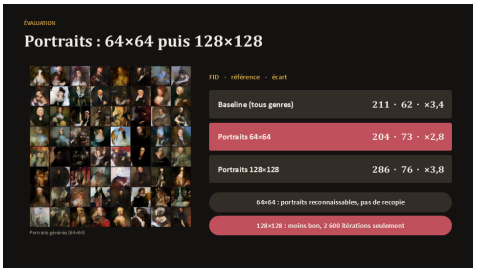
Notre toute première version, entraînée sur moins de 3 000 images, montrait un damier et un début de mode collapse. Avec deux fois plus d'images et sept fois plus d'itérations, sans rien changer d'autre, le FID passe de 335 à 211 et le KID est divisé par deux. Mais les images restaient abstraites.



20. Un jeu homogène : les portraits

Pape

Notre hypothèse : le jeu était trop hétérogène, avec 10 genres et 7 siècles. Nous avons donc constitué un jeu de portraits seulement : 4 068 sélectionnés, le maximum du catalogue, 3 737 récupérés. Avant d'entraîner, nous avons vérifié 16 images à la sortie de notre pipeline : bien cadrées, sans déformation, avec des valeurs entre −1 et 1.

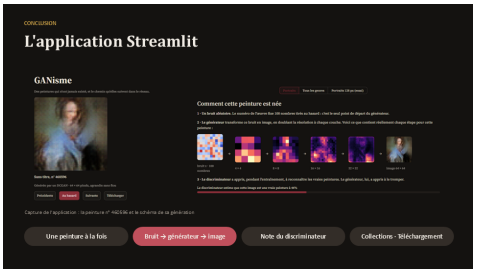


21. Portraits : 64×64 puis 128×128

Pape

L'hypothèse est confirmée. Avec le même modèle et le même nombre d'itérations, les portraits sont reconnaissables : silhouette centrée, visage clair, col blanc. Le FID est à 2,8 fois la référence, contre 3,4 pour la baseline.

En 128×128, une époque prenait près de 5 minutes sur processeur : avec deux fois moins d'itérations, le résultat reste moins bon. Un modèle plus gros demande plus d'entraînement, idéalement sur GPU.



22. L'application Streamlit

Pape

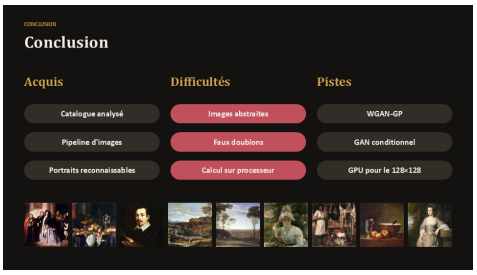
Notre application Streamlit présente une peinture à la fois. Chaque numéro redonne toujours la même œuvre. À côté, un schéma montre comment elle est née : les 100 nombres de départ, ce que contient le générateur à chaque couche, puis l'image finale, et enfin la note que lui donne le discriminateur. Nous pouvons vous en faire une démonstration.



23. Organisation

Pape

Nous avons mené ce projet à deux, avec notre code versionné sur GitHub. [À compléter : outil de suivi des tâches, fréquence de nos points d'équipe.]



24. Conclusion

Pape

Pour conclure : nous avons construit une chaîne complète, de l'analyse du catalogue à une application. Notre principal enseignement est que la qualité des données compte autant que le modèle : en rendant le jeu homogène, nous sommes passés d'images abstraites à des portraits reconnaissables. Pour aller plus loin, nous testerions le WGAN-GP, un GAN conditionnel et un entraînement en 128×128 sur GPU.

Merci pour votre attention. Nous sommes prêts à répondre à vos questions.
